SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence COURSE CODE: CSA1741

CO1-AT1-Analytical Problem Solving

Register Number	**192319056**
Name	**THARIKA SHREE M**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

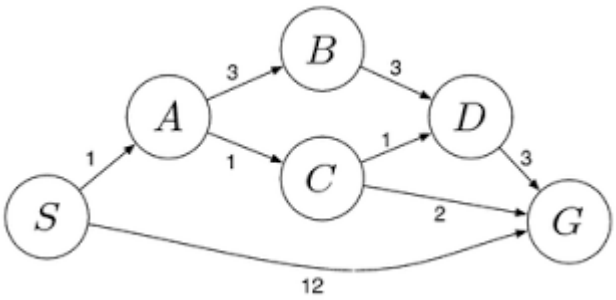
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path

from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I **Tharika Shree M (192319056)** *(Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tharika Shree M

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

1. Water Jug Problem (10 Marks)

Problem statement

Two jugs are available: a 4-gallon jug and a 3-gallon jug. Neither jug has measurement markings. The objective is to obtain exactly 2 gallons of water in the 4-gallon jug using only the pump, the ground, and the two jugs.

Problem formulation

1. Initial state: (0, 0), where the first value represents water in the 4-gallon jug and the second represents water in the 3-gallon jug.
2. Goal state: Any state in which the 4-gallon jug contains exactly 2 gallons, such as (2, 0).
3. Operators: Fill either jug, empty either jug, or pour water from one jug into the other until the source is empty or the destination is full.
4. Constraints: The 4-gallon jug can hold at most 4 gallons, and the 3-gallon jug can hold at most 3 gallons.

Step-by-step solution

Step	Action	4-gallon jug	3-gallon jug
1	Start with both jugs empty	0	0
2	Fill the 3-gallon jug	0	3
3	Pour the 3-gallon jug into the 4-gallon jug	3	0
4	Fill the 3-gallon jug again	3	3
5	Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full	4	2
6	Empty the 4-gallon jug onto the ground	0	2
7	Pour the remaining 2 gallons into the 4-gallon jug	2	0

Final answer: The 4-gallon jug contains exactly 2 gallons of water.

2. Mars Rover as an Intelligent Agent (10 Marks)

A Mars Rover is an autonomous robotic vehicle designed to explore the Martian surface, collect samples, analyze rocks and soil, and transmit scientific data to Earth. It uses sensors to perceive its surroundings and actuators to perform actions.

i. What are the percepts for this agent?

Percepts are the information received by an intelligent agent through its sensors.

The Mars Rover receives the following percepts:

1. Camera images: Images of rocks, soil, hills, and nearby obstacles.
2. Distance measurements: Information about the distance to rocks, craters, and other obstacles.

3. Position and orientation: Location, direction, and movement information obtained from navigation sensors.
4. Environmental conditions: Temperature, radiation, dust, and atmospheric conditions.
5. Scientific measurements: Chemical composition, mineral content, and other properties of collected samples.
6. System status: Battery level, wheel condition, instrument health, and communication status.

These percepts help the Rover understand its surroundings and make appropriate decisions.

ii. Characterize the operating environment

The Mars Rover operates in the following environment:

Environment property	Characterization	Explanation
Observability	Partially observable	Sensors cannot detect every environmental condition.
Determinism	Stochastic	Wheel slippage, dust, and unexpected obstacles introduce uncertainty.
Episodic or sequential	Sequential	Each movement affects future decisions and available routes.
Static or dynamic	Dynamic in practice	Dust, changing light, and environmental conditions may change during operation.
Discrete or continuous	Hybrid	Locations and actions may be represented discretely, while position, speed, and temperature are continuous.
Number of agents	Primarily single-agent	The Rover makes its own decisions, although communication with Earth and coordination with other systems may assist it.
Knowledge	Initially incomplete	The Rover must discover and update information about unfamiliar terrain.

iii. What actions can the agent take?

The Mars Rover can perform several actions:

1. Movement: Move forward, reverse, turn, or stop.
2. Navigation: Select a route and avoid rocks, craters, and steep slopes.
3. Sample collection: Extend a robotic arm, drill rocks, or collect soil samples.
4. Scientific analysis: Operate instruments to examine the composition of samples.
5. Communication: Transmit images, measurements, and scientific results to Earth.
6. Energy management: Enter a low-power mode or return to a safer location when battery levels are low.
7. System protection: Stop movement when a fault or dangerous condition is detected.

iv. How can the performance of the agent be evaluated?

The Rover's performance can be measured using a performance measure that evaluates how successfully it achieves its mission objectives.

- Scientific success: Number and quality of useful samples collected and analyzed.
- Navigation efficiency: Distance travelled, time taken, and successful arrival at target locations.
- Safety: Ability to avoid obstacles, prevent damage, and maintain system reliability.
- Energy efficiency: Amount of battery power used to complete tasks.
- Data quality: Accuracy and usefulness of scientific measurements.
- Communication: Successful transmission of collected data to Earth.
- Mission completion: Percentage of assigned scientific objectives completed within the available time and resources.

A good Rover should maximize scientific value while minimizing risk, energy consumption, and unnecessary travel.

v. Most suitable agent architecture and justification

A model-based, goal-based agent with utility-based decision-making and learning capabilities is most suitable for the Mars Rover.

1. Model-based component: Maintains an internal map of the terrain, records obstacles, and remembers previously explored areas.
2. Goal-based component: Plans actions to reach scientific targets, collect samples, and complete mission objectives.
3. Utility-based component: Chooses among alternative routes by balancing scientific value, travel time, battery consumption, and safety.
4. Learning component: Uses previous observations and experiences to improve navigation and sample selection.
5. Reflex component: Immediately stops or changes direction when sensors detect a dangerous obstacle.

For example, if a rock sample has high scientific value but lies near a steep slope, the Rover can evaluate alternative routes and select the safest route that satisfies its energy constraints.

Conclusion

The Mars Rover requires a hybrid intelligent agent because it operates in an unfamiliar, partially observable, and uncertain environment. Combining internal state tracking, goal-directed planning, utility-based decisions, and immediate safety responses improves its autonomy, efficiency, and scientific performance.

3. Formulate the 8-Queens Problem Using Search Strategies

Problem statement

The 8-Queens problem requires placing eight queens on an 8×8 chessboard so that no two queens attack each other.

A queen can attack another queen in the same row, column, or diagonal. Therefore, a valid solution must ensure that no two queens share any of these positions.

Problem formulation

1. Initial state

An empty chessboard with no queens placed.

2. State representation

Here, represents the row of the queen placed in column and is the number of queens placed so far.

For example, means that three queens are placed in columns 1, 2, and 3, in rows 2, 4, and 1, respectively.

3. Actions

Place a queen in an unoccupied row of the next column, provided it does not attack any previously placed queen.

4. Transition model

The transition model adds one queen to the next column if the selected position is safe.

5. Goal test

The goal is reached when eight queens have been placed without any pair attacking each other.

6. Path cost

If every queen placement has a cost of one, the total cost of reaching a complete arrangement is eight. In the usual backtracking formulation, the main objective is to find a valid solution rather than minimize this cost.

Search strategy: Backtracking search

Backtracking is an effective depth-first search technique for solving the 8-Queens problem.

Procedure:

1. Start with an empty chessboard.
2. Place a queen in a safe row of the first column.
3. Move to the next column and place another queen in a safe row.
4. Continue until all eight queens are placed.
5. If no safe position is available, return to the previous column and move that queen to another row.
6. Repeat until a complete valid arrangement is found.

Pseudocode

```
def solve_queens(board, col):
    if col == 8:
        return True

    for row in range(8):
        safe = True

        for c in range(col):
            if board[c] == row:
                safe = False
                break

            if abs(board[c] - row) == abs(c - col):
                safe = False
                break
```

```
            if safe:
                board[col] = row

                if solve_queens(board, col + 1):
                    return True

                board[col] = -1  # Backtrack

        return False


board = [-1] * 8

if solve_queens(board, 0):
    print("Queen positions by column:", board)
else:
    print("No solution found")
```

The program returns one valid arrangement, with each value representing a row numbered from 0 to 7.

Other search strategies

Search strategy	Application to 8-Queens	Limitation
Breadth-First Search (BFS)	Explores arrangements level by level.	Requires large amounts of memory.
Depth-First Search (DFS)	Explores one arrangement until it fails or succeeds.	Can explore unproductive branches deeply.
Backtracking	Rejects unsafe partial arrangements immediately.	May still explore many alternatives.
Heuristic search	Can prioritize promising placements using conflict information.	Requires a suitable heuristic and may need additional optimization.

Conclusion

The 8-Queens problem is a constraint-satisfaction and state-space search problem. Backtracking with one queen per column is efficient because it eliminates unsafe arrangements early rather than generating every possible board configuration.

4. OLA Cab Booking Using a Problem-Solving Agent (10 Marks)

Problem statement

A customer wants to travel from a starting location to a destination using the OLA Cab booking application. The customer can select a suitable cab type, such as Mini, Micro, Sedan, Shared, or Prime, depending on comfort, cost, and availability.

Type of problem-solving agent

A goal-based problem-solving agent is suitable because it identifies the customer's destination as the goal, evaluates available cab options, and selects an appropriate action to complete the trip.

A utility-based component can improve the decision by comparing fare, waiting time, comfort, and travel preferences.

Problem formulation

- Initial state: Customer's current location.
- Goal state: Customer reaches the requested destination.
- Actions: Enter locations, select a cab type, view available cabs, book a cab, and begin the trip.
- Transition model: Each action changes the booking state, such as moving from location entry to cab selection and then to confirmed booking.
- Path cost: Estimated fare, waiting time, travel time, and other relevant costs.
- Goal test: The customer reaches the destination successfully.

Pseudocode for OLA Cab Booking

```
BEGIN

  INPUT pickup_location
  INPUT destination

  IF pickup_location = destination THEN
    DISPLAY "Pickup and destination must differ"
    STOP
  END IF

  INPUT preferred_cab_type

  available_cabs = GET_AVAILABLE_CABS(
    pickup_location,
    destination
  )

  IF available_cabs IS EMPTY THEN
    DISPLAY "No cabs available"
    STOP
  END IF

  DISPLAY available_cabs

  selected_cab = FIND_CAB(
    available_cabs,
    preferred_cab_type
  )

  IF selected_cab IS NOT AVAILABLE THEN
```

```
    DISPLAY "Preferred cab unavailable"
    ASK customer to choose another cab
    selected_cab = GET_CUSTOMER_SELECTION()
  END IF

  estimated_fare = CALCULATE_FARE(
    selected_cab,
    pickup_location,
```

Explanation of the pseudocode

1. Input collection: The customer enters the pickup location and destination.
2. Validation: The application checks whether the entered locations are valid.
3. Cab availability: The system retrieves cabs available for the requested journey.
4. Cab selection: The customer chooses a cab based on preference, fare, comfort, and availability.
5. Fare calculation: The system estimates the trip cost and displays it before confirmation.
6. Booking: After the customer's confirmation, the application attempts to book the selected cab.
7. Trip tracking: The system provides driver information and tracks the journey until the destination is reached.

Conclusion:

A goal-based problem-solving agent is appropriate for the OLA booking process because it follows a sequence of actions to achieve the destination goal. Adding utility-based decision-making enables the application to recommend suitable cab options according to customer preferences, cost, comfort, and waiting time.

5. Uniform Cost Search (UCS) for a Delivery Network (10 Marks)

Problem statement

A logistics company needs to transport packages from a starting warehouse \(S\) to a destination warehouse \(G\) at minimum total cost. Each route between warehouses has a cost based on fuel consumption and travel time.

Uniform Cost Search (UCS) is used to find the least-cost path by expanding the node with the smallest accumulated path cost.

Problem formulation

- Initial state: Starting warehouse \(S\).
- Goal state: Destination warehouse \(G\).
- States: All warehouses in the delivery network.
- Actions: Travel from the current warehouse to any directly connected warehouse.
- Transition model: Moving along an edge leads to the connected warehouse.
- Path cost: Sum of the costs of all routes travelled.
- Goal test: Check whether the selected warehouse is \(G\).

Working of UCS

1. Initialize the starting warehouse \(S\) with a cost of zero.
2. Insert \(S\) into a priority queue.
3. Remove the warehouse with the smallest accumulated path cost.
4. If it is the goal warehouse \(G\), stop and return the path and total cost.
5. Otherwise, examine its neighbouring warehouses.
6. Calculate the new path cost for each neighbour.
7. Update the priority queue whenever a cheaper route to a warehouse is found.
8. Repeat until \(G\) is reached or no reachable warehouses remain.

Pseudocode

```
import heapq
def uniform_cost_search(graph, start, goal):
   priority_queue = [(0, start, [start])]
   best_cost = {start: 0}

   while priority_queue:
      cost, node, path = heapq.heappop(priority_queue)

      if cost > best_cost.get(node, float("inf")):
         continue

      if node == goal:
         return path, cost

      for neighbour, edge_cost in graph.get(node, []):
         new_cost = cost + edge_cost

         if new_cost < best_cost.get(
            neighbour, float("inf")
         ):
            best_cost[neighbour] = new_cost

            heapq.heappush(
               priority_queue,
               (new_cost, neighbour,
                path + [neighbour])
            )

   return None, float("inf")
```

Advantages of UCS

- Finds the optimal least-cost path when route costs are non-negative and standard UCS conditions hold.
- Considers the total accumulated cost rather than the number of edges.
- Works well for networks where different routes have different costs.
- Can avoid more expensive routes even if they contain fewer connections.

Limitations of UCS

- May explore many warehouses before reaching the destination.
- Requires memory to store the priority queue and cost information.
- Can be computationally expensive for large delivery networks.
- Does not use heuristic information to guide the search toward the destination.

Conclusion

Uniform Cost Search is suitable for the logistics problem because it selects routes based on accumulated cost and returns the least-cost path to the destination. This helps the company reduce transportation expenses and improve delivery planning.

Important: The actual weighted graph for Question 5 is not visible in your pasted assessment. The example above demonstrates the method, but its path and cost are not the answer to your specific graph. Upload the graph image or provide the connections and edge costs to calculate the exact UCS solution.